PELUQUERIA_DB

Descripción del proyecto:

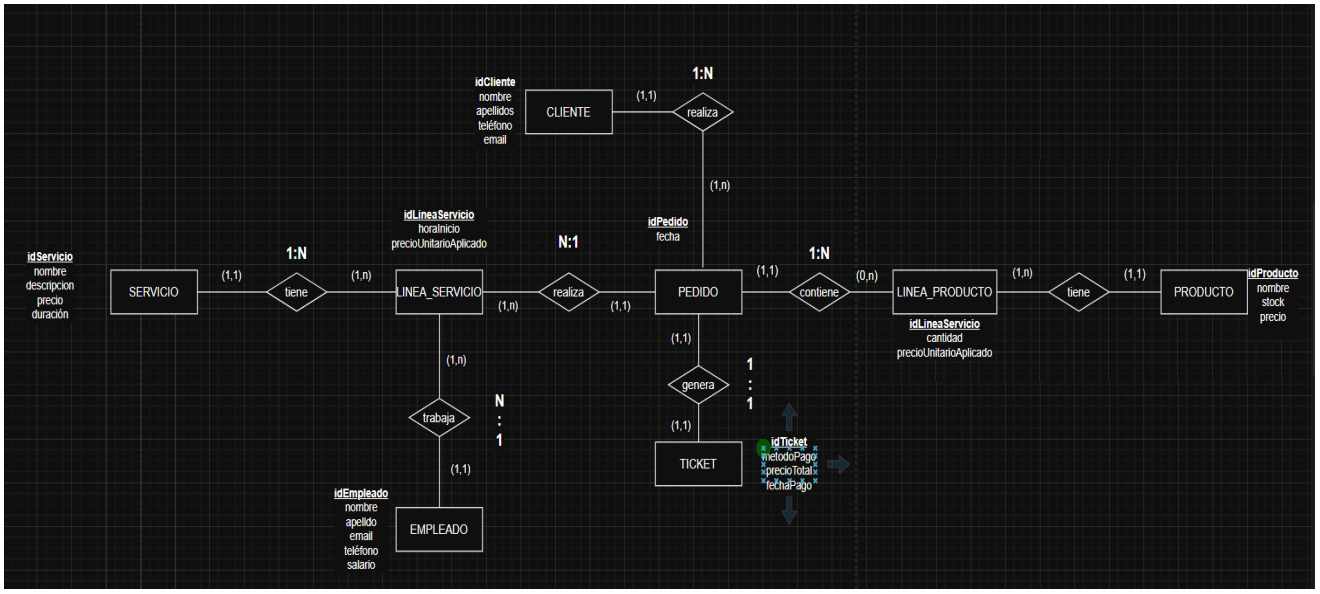
Este proyecto trata sobre el diseño y creación de una base de datos para gestionar el funcionamiento de una peluquería. La idea es poder almacenar toda la información importante del negocio de forma organizada, como los clientes, empleados, servicios que se realizan, productos que se venden y los pedidos o citas que se reservan.

Cada vez que un cliente pide cita, se genera un pedido donde se guarda la fecha de la reserva. A partir de ahí se registran los servicios realizados, indicando el empleado que los hace, la fecha y hora en la que se llevan a cabo y el precio aplicado. También se pueden añadir productos vendidos en ese mismo pedido.

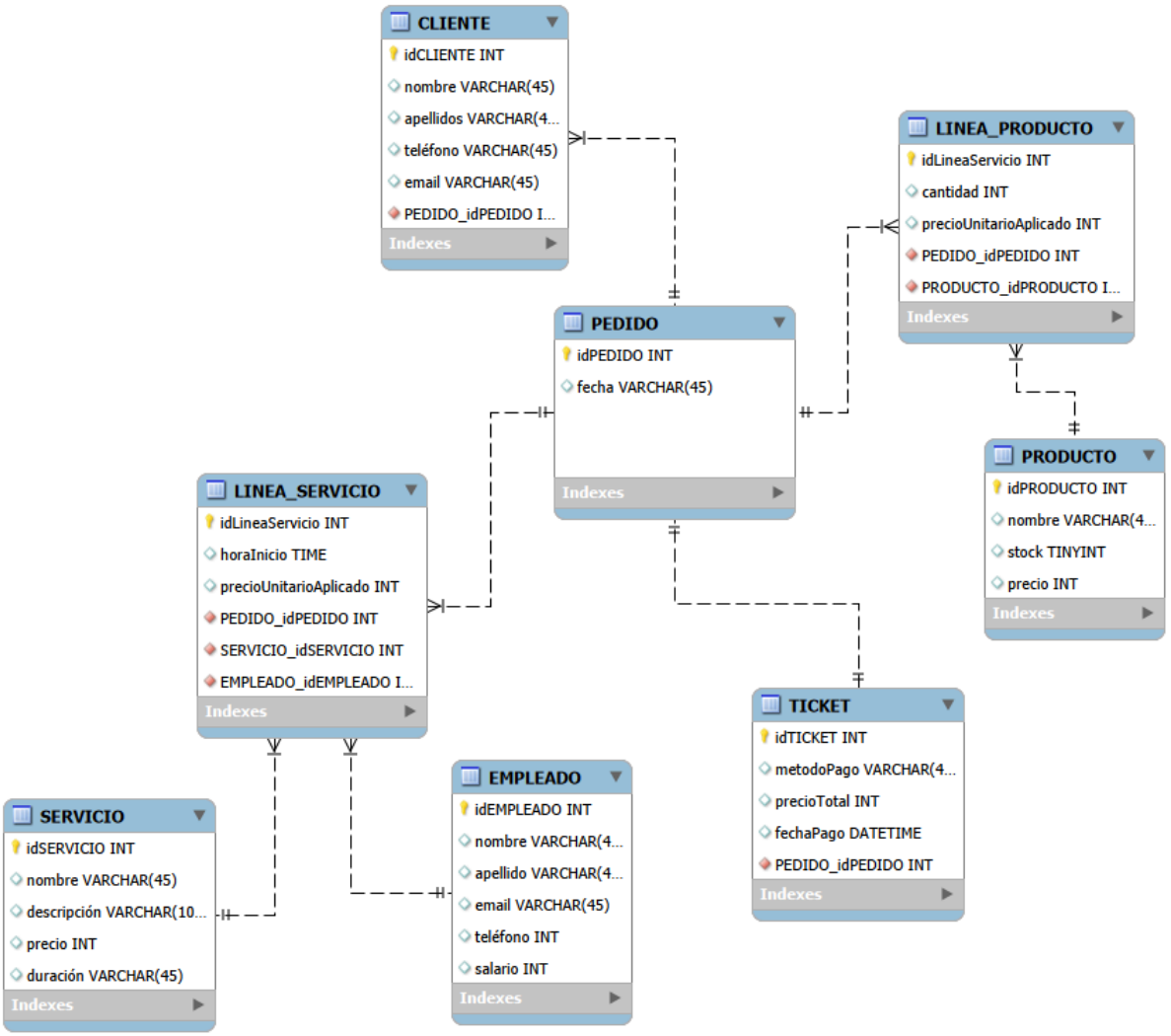
Por último, toda la información económica se recoge en el ticket, donde aparece el total pagado, el método de pago y la fecha en la que se realiza el cobro.

Con esta base de datos se pueden hacer consultas para analizar el negocio, como saber qué servicios se realizan más, cuánto factura cada empleado o cuánto gasta cada cliente.

MODELO MER CON DIAGRAMS:



MODELO RELACIONAL Y PASO A TABLAS:




```
DROP SCHEMA IF EXISTS `PeluqueriaDB`;
CREATE SCHEMA `PeluqueriaDB` DEFAULT CHARACTER SET utf8mb4;
USE `PeluqueriaDB`;

CREATE TABLE `CLIENTE` (
 `idCLIENTE` INT NOT NULL AUTO_INCREMENT,
 `nombre` VARCHAR(45) NOT NULL,
 `apellidos` VARCHAR(45) NULL,
 `telefono` VARCHAR(20) NULL,
 `email` VARCHAR(80) NULL,
 PRIMARY KEY (`idCLIENTE`),
 UNIQUE KEY `uk_cliente_email` (`email`)
) ENGINE=InnoDB;
CREATE TABLE `EMPLEADO` (
 `idEMPLEADO` INT NOT NULL AUTO_INCREMENT,
 `nombre` VARCHAR(45) NOT NULL,
```

```sql
 `apellido` VARCHAR(45) NULL,
 `email` VARCHAR(80) NULL,
 `telefono` VARCHAR(20) NULL,
 `salario` DECIMAL(10,2) NULL,
 PRIMARY KEY (`idEMPLEADO`),
 UNIQUE KEY `uk_empleado_email` (`email`)
) ENGINE=InnoDB;
CREATE TABLE `SERVICIO` (
 `idSERVICIO` INT NOT NULL AUTO_INCREMENT,
 `nombre` VARCHAR(45) NOT NULL,
 `descripcion` VARCHAR(100) NULL,
 `precio` DECIMAL(10,2) NULL,
 `duracionMin` INT NULL,
 PRIMARY KEY (`idSERVICIO`)
) ENGINE=InnoDB;
CREATE TABLE `PRODUCTO` (
 `idPRODUCTO` INT NOT NULL AUTO_INCREMENT,
 `nombre` VARCHAR(45) NOT NULL,
 `stock` INT NULL,
 `precio` DECIMAL(10,2) NULL,
 PRIMARY KEY (`idPRODUCTO`)
) ENGINE=InnoDB;

CREATE TABLE `PEDIDO` (
 `idPEDIDO` INT NOT NULL AUTO_INCREMENT,
 `fecha` DATE NOT NULL,
 `idCLIENTE` INT NULL,
 PRIMARY KEY (`idPEDIDO`),
 INDEX `idx_pedido_cliente` (`idCLIENTE`),
 CONSTRAINT `fk_pedido_cliente`
   FOREIGN KEY (`idCLIENTE`)
   REFERENCES `CLIENTE` (`idCLIENTE`)
   ON DELETE SET NULL
   ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE `TICKET` (
 `PEDIDO_idPEDIDO` INT NOT NULL,
 `metodoPago` VARCHAR(45) NULL,
 `precioTotal` DECIMAL(10,2) NULL,
 `fechaPago` DATETIME NULL,
 PRIMARY KEY (`PEDIDO_idPEDIDO`),
 CONSTRAINT `fk_ticket_pedido`
   FOREIGN KEY (`PEDIDO_idPEDIDO`)
   REFERENCES `PEDIDO` (`idPEDIDO`)
   ON DELETE CASCADE
   ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE `LINEA_SERVICIO` (
 `idLineaServicio` INT NOT NULL AUTO_INCREMENT,
 `horaInicio` DATETIME NULL,
```

```
`precioUnitarioAplicado` DECIMAL(10,2) NULL,
`PEDIDO_idPEDIDO` INT NOT NULL,
`SERVICIO_idSERVICIO` INT NOT NULL,
`EMPLEADO_idEMPLEADO` INT NOT NULL,
PRIMARY KEY (`idLineaServicio`),
INDEX `idx_ls_pedido` (`PEDIDO_idPEDIDO`),
INDEX `idx_ls_servicio` (`SERVICIO_idSERVICIO`),
INDEX `idx_ls_empleado` (`EMPLEADO_idEMPLEADO`),
CONSTRAINT `fk_ls_pedido`
  FOREIGN KEY (`PEDIDO_idPEDIDO`)
  REFERENCES `PEDIDO` (`idPEDIDO`)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
CONSTRAINT `fk_ls_servicio`
  FOREIGN KEY (`SERVICIO_idSERVICIO`)
  REFERENCES `SERVICIO` (`idSERVICIO`)
  ON DELETE RESTRICT
  ON UPDATE CASCADE,
CONSTRAINT `fk_ls_empleado`
  FOREIGN KEY (`EMPLEADO_idEMPLEADO`)
  REFERENCES `EMPLEADO` (`idEMPLEADO`)
  ON DELETE RESTRICT
  ON UPDATE CASCADE
) ENGINE=InnoDB;
CREATE TABLE `LINEA_PRODUCTO` (
`idLineaProducto` INT NOT NULL AUTO_INCREMENT,
`cantidad` INT NOT NULL DEFAULT 1,
`precioUnitarioAplicado` DECIMAL(10,2) NULL,
`PEDIDO_idPEDIDO` INT NOT NULL,
`PRODUCTO_idPRODUCTO` INT NOT NULL,
PRIMARY KEY (`idLineaProducto`),
INDEX `idx_lp_pedido` (`PEDIDO_idPEDIDO`),
INDEX `idx_lp_producto` (`PRODUCTO_idPRODUCTO`),
CONSTRAINT `fk_lp_pedido`
  FOREIGN KEY (`PEDIDO_idPEDIDO`)
  REFERENCES `PEDIDO` (`idPEDIDO`)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
CONSTRAINT `fk_lp_producto`
  FOREIGN KEY (`PRODUCTO_idPRODUCTO`)
  REFERENCES `PRODUCTO` (`idPRODUCTO`)
  ON DELETE RESTRICT
  ON UPDATE CASCADE
) ENGINE=InnoDB;
```

CONSULTAS Y VISTAS:

```
-- 1.Obtener la facturación mensual por empleado mostrando: id empleado, nombre completo, mes, número de servicios realizados,
-- número de pedidos atendidos, clientes distintos y el total facturado en servicios.
SELECT
 e.idEMPLEADO,
 CONCAT(e.nombre, ' ', e.apellido) AS empleado,
 DATE_FORMAT(ls.horainicio, '%Y-%m') AS mes,
 COUNT(*) AS num_servicios_realizados,
 COUNT(DISTINCT p.idPEDIDO) AS num_pedidos,
 COUNT(DISTINCT p.idCLIENTE) AS clientes_distintos,
 SUM(ls.precioUnitarioAplicado) AS total_servicios_eur
FROM LINEA_SERVICIO ls
INNER JOIN EMPLEADO e ON e.idEMPLEADO = ls.EMPLEADO_idEMPLEADO
INNER JOIN PEDIDO p ON p.idPEDIDO = ls.PEDIDO_idPEDIDO
INNER JOIN CLIENTE c ON c.idCLIENTE = p.idCLIENTE
INNER JOIN TICKET t ON t.PEDIDO_idPEDIDO = p.idPEDIDO
GROUP BY e.idEMPLEADO, empleado, mes
ORDER BY mes DESC, total_servicios_eur DESC;
```

SELECT e.idEMPLEADO, CONCAT(e.nombre, ' ', e.apellido) — Enter a SQL expression to filter results (use Ctrl+Space)

	idEMPLEADO	empleado	mes	num_servicios_realizados	num_pedidos	clientes_distintos	total_servicios_eur
1	5	Laura Díaz Muñoz	2026-02	40	19	19	844
2	2	Javier López Jiménez	2026-02	28	14	14	640
3	10	Laura Rodríguez Hernández	2026-02	19	9	8	417
4	4	Lucía Muñoz González	2026-02	19	8	8	413
5	9	Lucía Díaz García	2026-02	20	12	12	411
6	8	Javier López Moreno	2026-02	18	8	8	406
7	3	Alejandro Jiménez Moreno	2026-02	13	7	7	378
8	6	Paula Jiménez Hernández	2026-02	17	8	8	374
9	7	Sara García Ruiz	2026-02	15	10	10	247
10	1	Daniel Gómez Rodríguez	2026-02	6	4	4	144
11	5	Laura Díaz Muñoz	2026-01	46	23	22	1.063

```
-- 2. Mostrar los 5 servicios más realizados indicando cuántas veces se han realizado y cuánto dinero han generado.
SELECT s.idSERVICIO, s.nombre AS servicio, COUNT(*) AS veces_realizado,
SUM(ls.precioUnitarioAplicado) AS ingresos_servicio_eur
FROM LINEA_SERVICIO ls
JOIN SERVICIO s ON s.idSERVICIO = ls.SERVICIO_idSERVICIO
GROUP BY s.idSERVICIO, s.nombre
ORDER BY veces_realizado DESC, ingresos_servicio_eur DESC
LIMIT 5;
```

	idSERVICIO	servicio	veces_realizado	ingresos_servicio_eur
1	6	Mechas	53	1.855
2	13	Decoloración	41	1.230
3	3	Lavado	38	190
4	12	Tratamiento capilar	37	740
5	8	Arreglo de barba	37	296

```
-- 3. Listar los clientes cuyo gasto total en la peluquería es superior al gasto medio de todos los tickets registrados.
SELECT c.idCLIENTE, CONCAT(c.nombre, ' ', c.apellidos) AS cliente, SUM(t.precioTotal) AS gasto_total
FROM CLIENTE c
INNER JOIN PEDIDO p ON p.idCLIENTE = c.idCLIENTE
INNER JOIN TICKET t ON t.PEDIDO_idPEDIDO = p.idPEDIDO
GROUP BY c.idCLIENTE, cliente
HAVING SUM(t.precioTotal) > (
 SELECT AVG(precioTotal)
 FROM TICKET
)
ORDER BY gasto_total DESC;
```

	idCLIENTE	cliente	gasto_total
1	559	Rubén Ramírez Hernández	336
2	538	David Molina Romero	225
3	436	Marta Muñoz Rodríguez	211
4	491	David Serrano Ortiz	199
5	125	Hugo Navarro Núñez	173
6	46	Miguel Moreno Ramos	168
7	511	Irene Pérez Domínguez	163
8	6	Alba Vázquez Jiménez	155
9	356	Adrián Hernández Romero	153
10	533	Marta Ramos Navarro	150
11	339	Iván Pérez Gil	148

```
-- 4. Mostrar el listado de tickets indicando: cliente, email (mostrar “N/A” si es NULL), método de pago, total y fecha de pago formateada.
SELECT t.PEDIDO_idPEDIDO AS idPedido, CONCAT(c.nombre, ' ', c.apellidos) AS cliente, IFNULL(c.email, 'N/A') AS email, t.metodoPago, t.precioTotal, DATE_FORMAT(t.fechaPago, '%d/%m/%Y %H:%i') AS fechaPago_formateada
FROM TICKET t
INNER JOIN PEDIDO p ON p.idPEDIDO = t.PEDIDO_idPEDIDO
INNER JOIN CLIENTE c ON c.idCLIENTE = p.idCLIENTE
ORDER BY t.fechaPago DESC;
```

	idPedido	cliente	email	metodoPago	precioTotal	fechaPago_formateada
1	242	Iván Pérez Gil	ivan.perez339@correo.es	Tarjeta	148	26/02/2026 12:43
2	244	Rubén Ramírez Hernández	ruben.ramirez559@outlook.com	Tarjeta	125	25/02/2026 13:10
3	238	Carlos Álvarez Suárez	carlos.alvarez237@hotmail.com	Tarjeta	66	24/02/2026 16:31
4	236	Carlos Rodríguez Molina	carlos.rodriguez281@outlook.com	Efectivo	8	24/02/2026 09:26
5	245	Paula Gutiérrez Alonso	paula.gutierrez687@proton.me	Efectivo	35	23/02/2026 19:09
6	234	Noelia Serrano Díaz	noelia.serrano540@correo.es	Efectivo	58	22/02/2026 16:51
7	235	David Molina Romero	david.molina538@hotmail.com	Bizum	110	22/02/2026 12:26
8	247	Patricia Gutiérrez Sánchez	patricia.gutierrez28@outlook.com	Efectivo	20	22/02/2026 11:20
9	232	David Navarro Morales	david.navarro456@gmail.com	Tarjeta	18	20/02/2026 18:18
10	237	Iván Serrano Blanco	ivan.serrano220@gmail.com	Tarjeta	45	20/02/2026 18:01
11	222	Laura Gutiérrez Moreno	laura.gutierrez425@correo.es	Tarjeta	30	20/02/2026 10:39

```sql
-- 5. Mostrar los pedidos en los que se han vendido productos indicando cliente, fecha de reserva, unidades totales vendidas y el importe total en productos.
SELECT p.idPEDIDO, CONCAT(c.nombre, ' ', c.apellidos) AS cliente, p.fecha AS fecha_reserva, SUM(lp.cantidad) AS unidades_totales, SUM(lp.cantidad * lp.precioUnitarioAplicado) AS total_productos_eur
FROM PEDIDO p
INNER JOIN CLIENTE c ON c.idCLIENTE = p.idCLIENTE
INNER JOIN LINEA_PRODUCTO lp ON lp.PEDIDO_idPEDIDO = p.idPEDIDO
GROUP BY p.idPEDIDO, cliente, p.fecha
ORDER BY total_productos_eur DESC;
```

	idPEDIDO	cliente	fecha_reserva	unidades_totales	total_productos_eur
1	182	Rubén Ortiz Romero	2026-01-30	3	99
2	140	Miguel Moreno Ramos	2026-01-23	3	93
3	15	Marta Serrano Suárez	2026-01-04	5	93
4	242	Iván Pérez Gil	2026-02-13	4	85
5	95	Rubén Navarro López	2026-01-15	3	84
6	84	Rubén Ramírez Hernández	2026-01-14	3	82
7	77	Hugo Navarro Núñez	2026-01-13	2	80
8	177	Adrián Hernández Romero	2026-01-29	3	80
9	217	Óscar González Gil	2026-02-07	3	76
10	209	Silvia Núñez Serrano	2026-02-05	3	75
11	156	Claudia Castro Rodríguez	2026-01-26	4	72

```sql
-- 6. Mostrar por empleado los minutos totales trabajados, número de servicios realizados e ingresos generados por servicios.
SELECT e.idEMPLEADO, CONCAT(e.nombre, ' ', e.apellido) AS empleado, SUM(s.duracionMin) AS minutos_totales, COUNT(*) AS num_servicios, SUM(ls.precioUnitarioAplicado) AS ingresos_servicios_eur
FROM LINEA_SERVICIO ls
INNER JOIN EMPLEADO e ON e.idEMPLEADO = ls.EMPLEADO_idEMPLEADO
INNER JOIN SERVICIO s ON s.idSERVICIO = ls.SERVICIO_idSERVICIO
GROUP BY e.idEMPLEADO, empleado
ORDER BY ingresos_servicios_eur DESC;
```

	idEMPLEADO	empleado	minutos_totales	num_servicios	ingresos_servicios_eur
1	5	Laura Díaz Muñoz	5.225	86	1.907
2	9	Lucía Díaz García	3.495	61	1.263
3	10	Laura Rodríguez Hernández	3.140	51	1.155
4	2	Javier López Jiménez	3.020	48	1.113
5	7	Sara García Ruiz	3.035	51	1.096
6	6	Paula Jiménez Hernández	2.660	49	1.028
7	8	Javier López Moreno	2.395	46	893
8	3	Alejandro Jiménez Moreno	2.440	34	858
9	1	Daniel Gómez Rodríguez	2.365	42	852
10	4	Lucía Muñoz González	2.070	38	785

```sql
-- 7. Mostrar la facturación diaria indicando método de pago, número de tickets y total facturado por día.
SELECT DATE(t.fechaPago) AS dia, t.metodoPago, COUNT(*) AS num_tickets,
SUM(t.precioTotal) AS total_dia_eur
FROM TICKET t
GROUP BY dia, t.metodoPago
ORDER BY dia DESC, total_dia_eur DESC;
```

	dia	metodoPago	num_tickets	total_dia_eur
1	2026-02-26	Tarjeta	1	148
2	2026-02-25	Tarjeta	1	125
3	2026-02-24	Tarjeta	1	66
4	2026-02-24	Efectivo	1	8
5	2026-02-23	Efectivo	1	35
6	2026-02-22	Bizum	1	110
7	2026-02-22	Efectivo	2	78
8	2026-02-20	Tarjeta	3	93
9	2026-02-19	Tarjeta	1	134
10	2026-02-18	Tarjeta	2	50
11	2026-02-17	Tarjeta	3	209

```sql
-- 8. Crear una vista que muestre el detalle de los tickets junto con los datos del cliente y la fecha de reserva.
CREATE OR REPLACE VIEW v_ticket_detalle AS
SELECT
 t.PEDIDO_idPEDIDO AS idPedido,
 p.fecha AS fecha_reserva,
 t.fechaPago,
 t.metodoPago,
 t.precioTotal,
 c.idCLIENTE,
 CONCAT(c.nombre, ' ', c.apellidos) AS cliente,
 c.telefono,
 c.email
FROM TICKET t
INNER JOIN PEDIDO p ON p.idPEDIDO = t.PEDIDO_idPEDIDO
INNER JOIN CLIENTE c ON c.idCLIENTE = p.idCLIENTE;
```

```sql
-- 9. Crear una vista que muestre la facturación mensual de cada empleado basada en los servicios realizados.
CREATE OR REPLACE VIEW v_facturacion_empleado_mes AS
SELECT
 e.idEMPLEADO,
 CONCAT(e.nombre, ' ', e.apellido) AS empleado,
 DATE_FORMAT(ls.horainicio, '%Y-%m') AS mes,
 COUNT(*) AS num_servicios_realizados,
 SUM(ls.precioUnitarioAplicado) AS total_servicios_eur
FROM LINEA_SERVICIO ls
INNER JOIN EMPLEADO e ON e.idEMPLEADO = ls.EMPLEADO_idEMPLEADO
GROUP BY e.idEMPLEADO, empleado, mes;
```

FUNCIONES

```
DELIMITER //
create function total_productos_vendidos(id int)
returns decimal(10,2)
deterministic
begin
  declare total decimal(10,2);
  select IFNULL(SUM(cantidad * precioUnitarioAplicado), 0)
  into total
  from linea_producto
  where idPEDIDO = id;
  return total;
end//
DELIMITER ;



DELIMITER //
create function total_servicios_pedido(id int)
returns decimal(10,2)
deterministic
BEGIN
  declare total decimal(10,2);
  select IFNULL(SUM(precioUnitarioAplicado), 0)
  into total
  from linea_servicio
  where idPEDIDO = id;
  return total;
end//
DELIMITER ;
```

PROCEDIMIENTOS

```
DELIMITER //
create procedure actualizar_total_ticket(in id int)
begin
  declare totalProductos decimal(10,2);
  declare totalServicios decimal(10,2);
        declare total decimal(10,2);
  set totalProductos = total_productos_pedido(id);
  set totalServicios = total_servicios_pedido(id);
  set total = totalProductos + totalServicios;
  update ticket
```

```
  set precioTotal = total
  where idPEDIDO = id;
end//
DELIMITER ;



DELIMITER //
create procedure resumen_pedido(in id int)
begin
  select
      idPEDIDO, CONCAT(c.nombre, ' ', c.apellidos) as cliente, fecha,
total_productos_pedido(idPEDIDO) as total_productos,
      total_servicios_pedido(idPEDIDO) as total_servicios,
total_productos_pedido(idPEDIDO) + total_servicios_pedido(idPEDIDO) AS
total_final
  from PEDIDO p inner join CLIENTE c
        ON c.idCLIENTE = p.idCLIENTE
  where idPEDIDO = id;
end//
DELIMITER ;
```

TRIGGERS

```
DELIMITER //
create trigger descontar_stock
after insert on LINEA_PRODUCTO
for each row
begin
  update PRODUCTO
  set stock = stock - new.cantidad
  where idPRODUCTO = new.idPRODUCTO;
end//
DELIMITER ;

DELIMITER $$
create trigger actualizar_ticket_producto
after insert on LINEA_PRODUCTO
for each row
begin
  call actualizar_total_ticket(NEW.idPEDIDO);
END//
DELIMITER ;
```

CONCLUSIÓN:
Con este proyecto he aprendido a diseñar y crear una base de datos relacional completa, relacionando tablas mediante claves y gestionando datos de clientes, servicios y ventas. También he practicado consultas multitabla, funciones y agrupaciones para obtener información útil.
Como mejora futura, se podría añadir una gestión más avanzada de citas y control de stock automático de productos.

ENLACE A GITHUB:
https://github.com/Mario-bits64/Peluqueria